THE PLAIN-ENGLISH PLAYBOOK

Drydock

A way of building things with Claude Code that holds together: plan, build, check, prove, remember.

For anyone who has handed Claude an idea, watched it build something that broke, fixed it, and watched twenty new problems appear. This book is the method that stops that, sized to fit you.

Version 0.1, October 2026. Companion to the Drydock plugin at github.com/wjp3/drydock. Two-page quickstart first; the full playbook after it. Read the quickstart, install, and start. Come back to the rest when you need it.

Drydock in two pages

What it is

Drydock is a set of habits, packaged as a Claude Code plugin, that make a build hold together. You describe what you want in your own words. Drydock interviews you, writes a map of small segments with a test that says when each is done, builds one segment at a time, has a *separate* reviewer check the work, attacks it, walks it like a person would, and then stops to show you the evidence. Nothing goes live without your go. You never have to read code unless you want to.

Install (about two minutes)

You need a paid Claude plan (Pro or above) and the Claude desktop app, plus Node.js 20 or newer, git, and the GitHub command-line tool `gh` for the apps you will build (if someone set up your computer for you, these are already there). Mac or Windows.

The plugin, with no terminal: open `claude.ai/customize/plugins` in your browser (or **Customize > Plugins** in the desktop app), choose **Add > Add marketplace**, type `wjp3/drydock`, and add it. Drydock is saved to your account and shows up in Claude Code the next time you start a session. If you were sent `drydock-plugin.zip` instead, choose **Add > Upload plugin** and pick that file.

Or in a Claude Code session, type:

```
/plugin marketplace add wjp3/drydock
/plugin install drydock@drydock
```

Then make a folder for your project (empty is fine), open it in the desktop app's Code tab, and type `/drydock:setup`. It checks your computer, asks four questions, and creates the files that will hold the build's memory.

Your first hour

1. **`/drydock:setup`**. Four questions: your Claude plan, how technical you are, new or existing project, what you are building in one sentence. It picks a size for the machinery (see Tiers) and offers a few companion tools.
2. **`/drydock:plan`**. The interview. Answer in plain words; say "I don't know" when you don't. You get `VISION.md` (what done means, in your words) and `BUILD-MAP.md` (the segments in order, each with its test). Then lenses you do not have review the map and propose fixes. You decide.
3. **`/drydock:build S0`**. The first segment is always "the skeleton is live": an empty app with its name at a web address, one passing test, code saved on GitHub. It proves the whole path from code to a URL before any feature exists, so the path is never the surprise.
4. **Try it. Say "next."** Every segment ends with a stop: what you can now do and where to try it, the test clause by clause with evidence, what the separate check found and what was fixed, what is not proven. If it does what you expected, say **next**. If not, say what is off.
5. **`/drydock:handoff`** when you stop for the day. **`/drydock:wake`** when you come back. Start each new segment in a fresh session.

The five verbs

Verb	You type	What happens
Plan	`/drydock:plan`	Interview, then the vision and the map, then a review of the map.
Build	`/drydock:build S3` or "next"	Freeze the segment's test. Write the checks first, failing. Build until green. Save a checkpoint. Then check, prove, and stop.
Check	`/drydock:check`	Earlier segments' tests run again. A fresh reviewer, on a different model, reads the change against its card and runs the tests itself. (Build does this for you.)
Prove	`/drydock:prove`	The red team tries inputs nobody tried. The running app is walked like a person would use it. A proof sheet is filed. (Build does this for you.)
Remember	`/drydock:wake`, `/drydock:handoff`, `/drydock:status`	Where we are, what is settled, what is next. Memory lives in files, not in the conversation.

Also: `/drydock:ship` (preview first, then live only on your yes), `/drydock:panel` (several expert lenses on a decision), `/drydock:crew` (the full build crew, for the heaviest segments), `/drydock:help`.

The rules that protect you

- **You decide.** Drydock recommends, every time, and shows its evidence. It never waits on you for things it can propose, and it never acts outside your computer without your go for that exact step.
- **Nothing is done on its say-so.** "Done" comes with the command that was run and what it printed, or a screenshot. A separate reviewer agreed. If you don't see evidence, ask for it.
- **Nothing leaves your computer without a yes.** No code pushed, nothing deployed, nothing published, until you say go for that step. A built-in gate asks even if Drydock forgets.
- **Stuck means stop.** After three honest attempts at the same problem, Drydock stops and tells you what is blocking, with options and a recommendation. It never pretends.
- **One segment per session.** Long conversations get worse. Hand off, start fresh, wake.

> ### Three sentences worth memorizing
>
> **"Show me the evidence."** Use it any time something is called done. You should get a command and its output, or a screenshot, not a summary.
> **"Stop and tell me what's blocking."** Use it when it keeps trying the same thing.
> **"Handoff."** Use it when the conversation has gotten long or confused. Then open a fresh session and type `/drydock:wake`.

What Drydock will not do

Create accounts or type passwords for you (you do those in your browser). Push, deploy, or publish without your yes. Call anything done without evidence. Keep going when it is stuck. Upgrade to the expensive machinery on its own.

THE PLAYBOOK

Contents

1. Why builds break
2. The method on one page
3. Setting up: accounts, tiers, companions
4. Planning: the interview, the vision, the map
5. Building a segment
6. Check and prove
7. Panels
8. The crew
9. Remembering
10. Shipping
11. When things go wrong
12. Glossary
Appendix A. Command reference
Appendix B. The files Drydock creates
Appendix C. Where the method comes from

1. Why builds break

If you have built with Claude before, you have probably met all four of these. Each has a fix, and the fix has to be structural, not a reminder in a prompt, because reminders fade as the conversation grows.

Failure one: no plan, so Claude improvises

You give it the idea. It starts coding. It makes a hundred small decisions you never saw, each reasonable alone, and the result is something that is not quite what you meant. Then you ask for a change, and the change fights the decisions underneath it.

The fix. Before any code, a written vision in your words and a map of small segments, each with a test that says when it is done. The plan is short enough to read and correct. A bad line of plan becomes hundreds of bad lines of code; a bad line of code is just a bad line of code. So the plan is where your attention pays most. Anthropic's own guidance says the same: have Claude interview you first and write the spec, then build from the spec in a fresh session.

Failure two: fixes break other things

You report a problem. It fixes that problem. Three other things stop working, because the fix touched code that other screens depended on and nothing checked them.

The fix. Tests written before the code, so every segment leaves a check behind. A look at what else touches the changed code before editing it. Every earlier segment's checks run again before any new "done". And a saved checkpoint every time a test goes green, so there is always a known-good version to go back to. "Main is the shelf of things that work."

Failure three: context lost between sessions

A new session starts cold. It re-decides things you settled last week, or forgets a feature exists, or builds a second version of something. Even inside one long session, quality drops as the conversation fills up; Claude starts forgetting earlier instructions.

The fix. Memory in files, not in the conversation. `STATE.md` says where we are. `DECIDED.md` says what is settled, with the reason, so it is never re-argued. Every session ends with a handoff note and starts with a wake-up that reads it. One segment per session, then a fresh start. A snapshot is saved automatically before Claude compresses a long conversation.

Failure four: Claude says done when it isn't

"All tests pass. The feature is complete." You try it. It doesn't work. Anthropic's researchers put it plainly: agents reliably skew positive when grading their own work. The builder is the wrong judge of the build.

The fix. The segment's test is frozen word for word before building, and graded by echo: the grader must repeat each clause exactly beside its verdict, so a softened or paraphrased bar shows up as a text difference, not a judgment call. A fresh reviewer, running on a different model so it does not share the builder's blind spots, re-runs the tests itself and reads the change against the card. A red team attacks with inputs the builder never tried. The running app is walked. The summary you read is audited against the evidence before it reaches you. And a built-in gate holds any "done" message while the test command is red.

> ### Where this comes from
>
> Drydock is a working protocol developed over several real products and then checked, on October 11, 2026, against Anthropic's published guidance and research and the main public methods (Superpowers, Spec Kit, BMAD, Anthropic's own feature-dev plugin, and others). Every method that works lands on the same five moves. Drydock adds three the public methods lack: a different model family checks the builder, the summary you read is audited first, and every control traces to a specific failure with a rule for retiring it. Appendix C lists everything that was read.

2. The method on one page

Five verbs. The first and last are about thinking and remembering. The middle three are the loop that builds.

Plan

An interview until a stranger could build the right thing from the vision alone. Then the map: segments in dependency order, each a card with what you will be able to do when it is done and a binary exit test. The first segment, S0, is always the skeleton. Then the map is reviewed by lenses you do not have (a real user on a bad day, an engineer who has been paged at 3 a.m., a contrarian) and you decide what changes.

Build

One segment. Its exit test is frozen. Each clause becomes a check that fails before the code exists. Build until the checks pass. Save a checkpoint at every green.

Check

Every earlier segment's checks run again. A fresh reviewer on a different model reads the change against the card, runs the tests itself, and reports gaps that affect correctness. Gaps get fixed. Style opinions do not.

Prove

A red team attacks the segment with inputs the builder never tried, both directions on every rule (the bad case is refused *and* the good case next to it still works). The running app is walked the way a person would use it. Evidence goes on a proof sheet.

Remember

`STATE.md` for where we are. `DECIDED.md` for what is settled. A handoff to close the session, a wake-up to open the next one.

The loop, drawn

```
card → freeze the exit test → write the failing checks → build → checks green →
checkpoint
     → earlier segments' checks again → fresh reviewer (other model) → fix gaps
     → red team → walk the running app → grade the frozen test by echo → proof
sheet
     → checkpoint, mark green, update STATE.md → STOP and show you
```

The stop at the end is the design. You try it. You say "next". The next segment starts in a fresh session.

Three rules that cut across everything

- **The one-sentence rule.** If a change can be described in one sentence and touches one place, skip the plan and the panel. Write the check, make it green, save. Ceremony is for things that deserve it.
- **Right-size the machinery.** Panels and the crew are earned by the build, not applied by default. A separate evaluator is worth its cost when the task is beyond what the model does reliably alone. That is Anthropic's finding and Drydock's rule.
- **Every control names its failure.** Each gate and each reviewer in Drydock says which of the four failures it exists to stop and when it can be retired. Models improve; a control that no longer catches anything should be removed, because every component in a harness encodes an assumption about what the model cannot do on its own.

3. Setting up

What you need on your computer

Tool	What it is for	Where to get it
A paid Claude plan	Claude Code needs Pro, Max, or Team.	claude.ai
The Claude desktop app	The thing you talk to. Its Code tab is Claude Code with a chat box instead of a terminal. It updates itself.	claude.com/download
Node.js 20+	Runs the app you build and Drydock's own small scripts.	nodejs.org, choose LTS
git	Saves checkpoints of your code.	git-scm.com (on a Mac, `xcode-select --install` also works)
gh	Talks to GitHub from the terminal.	cli.github.com, then `gh auth login`

Setup checks all of these and tells you exactly which is missing. If someone is setting up your computer for you, give them the *Drydock setup checklist* that comes with this playbook: a one-line installer puts everything in place and signs you in through your browser, and after that you never need a terminal. On Windows, Drydock's scripts are plain Node and run the same way.

Accounts you create yourself

Drydock never creates accounts or types passwords. You do these in your browser, and only when the build needs them.

Service	Why	When
GitHub	Where your code lives. Also where the Drydock plugin comes from.	Before So
Netlify	Hosts your app and gives every change a private preview address before anything goes live.	Before So, if the thing you build has a web address
Supabase	Database and sign-in, when your app needs to remember things between visits or let people log in.	When the plan says so

All three have free tiers that are real. Keys and passwords never go into the chat or into code. A file called `docs/SECRETS.md` says where each lives, never what it is.

The four setup questions

1. **Your Claude plan.** This picks the size of the machinery. If you don't know, say so; the light version is the default.
2. **How technical you are.** "I build by describing what I want and don't read code" is a fine answer and sets the voice: plain words, every term explained once, no code in your face unless you ask.
3. **New or existing project.**
4. **What you're building, in one sentence,** and a short name.

Tiers: the method at three sizes

Multi-agent runs cost many times what a conversation costs. On a Pro plan, a full panel can burn a week of usage in an afternoon. So the method never changes, but the machinery scales.

	lite	standard	full
Default when	Pro, or unknown	Max or Team; no other people's data, logins, or money	Max or Team; logins, other people's data, payments, or real users depending on it
Map review	one map critic	3-seat panel, 1 round	5 to 7 seats, up to 2 rounds
Per segment	a checker, a red team pass; you walk it	checker, red team, a walker, an honesty audit of the summary	same, plus the crew on marked segments
Agent runs per segment	2 to 4	4 to 7	15 to 25 when the crew runs

Drydock tells you before any large run and never upgrades the tier on its own. You can change it any time by asking: "switch Drydock to standard."

Companion tools (what each one is for)

Tool	In plain words
Context7	Gives Claude the current manual for the libraries it is using, so it stops guessing at how things work.
Netlify tools	Lets Claude create the preview address and publish, when you say go.
Supabase tools	Lets Claude set up and change your database, scoped to your one project.

Playwright	A robot browser. Lets the walker and the red team click through your running app and take screenshots.
typescript-lsp	Only if your project uses TypeScript: shows type errors after every edit.

Already built into Claude Code, nothing to install: `/code-review` (a fresh-context bug review), `/verify` (runs the app and checks a change against it), `/goal` (a separate evaluator keeps a session working until a condition holds), `/rewind` (go back to an earlier point). Drydock uses these where they fit.

> **One method at a time**
>
> Do not install other method plugins alongside Drydock (spec kits, agile-team packs, loop runners). Two methods that both want to own planning and the definition of done will fight. Pick one.

4. Planning: the interview, the vision, the map

The interview

Drydock asks until it could write every part of the vision without guessing, or about eight rounds, whichever comes first. It uses multiple-choice questions where there is a sensible default and open questions where it needs your story. It is told not to ask what it can infer and to dig into the parts you have not thought about. Roughly, it will ask about:

1. **Who** uses it, and the moment they first open it.
2. **The core loop**: the one thing a person does again and again, and what they see after.
3. **Day one versus someday**: what must exist for one real person to use it for the real purpose.
4. **Data**: does anything need to be remembered? Whose is it? Who may see it? Children? Health, money, legal?
5. **Logins**: who signs in, how many kinds of people, what one kind must never be able to do.
6. **Devices and places**: phones, bad wifi, shared computers.
7. **Must-nevers**: the things that would make you pull the plug.
8. **What already exists**: spreadsheets, forms, a logo.
9. **Constraints**: deadline, hours per week, money, rules you must follow.
10. **Done**, in your words: "When I can ____, this is finished."

> **How to answer**
>
> In your own words, with the real example in mind: "Mrs. Alvarez has 26 fourth graders and forty minutes" beats "a teacher with a class". Say "I don't know" freely; Drydock will propose a default and you can correct it. If you get tired of questions, say "just build it": Drydock writes the vision with its assumptions listed and you cross out the wrong ones. Proposing beats interrogating.

VISION.md

What done means, in your words. One paragraph of promise. The day-one story. Two finish lines: **MVP** (the smallest thing a real person can use for the real purpose) and **Finished** (everything in the vision). Must-nevers. What is out of scope on purpose. The assumptions Drydock made, for you to correct. Edit it whenever your mind changes, and tell Drydock you did.

The stack, in plain words

The "stack" is the set of tools the app is made from. Drydock picks the simplest one that meets the vision and writes the choice and the reason in `DECIDED.md`.

If the vision needs	Drydock picks	Because
Pages to read, no logins, nothing saved	A static site on Netlify	Nothing to secure, nothing to pay for, almost nothing can break.
People sign in and their things are saved	A web app (Vite and React) with Supabase for the database and logins, hosted on Netlify	Real free tiers; one database with built-in sign-in and per-person permissions; the most documented path for building with Claude.
Many people on phones	The same web app, made installable to the home screen	One codebase, no app store.
Money changes hands	Payments added in test mode first	Payments are where "it works" must be proven, not believed.

The map

`BUILD-MAP.md` is the contract. Segments in the order they must be built. Each is a card:

```
## S3 — Students can log a book
- Status: todo
- Mode: SOLO (one screen, one table, no new permissions)
- Size: M
- Depends on: S1, S2
- What you can do when it's done: A signed-in student picks a book and taps Log.
  It appears in their list with today's date.
- Exit test:
  - [ ] Logging a book as a signed-in student adds one row to the reading log
        with that student's id and today's date
  - [ ] The logged book appears at the top of the student's list without a page
reload
  - [ ] A signed-out visitor who asks the server to log a book is refused and
nothing is saved
- How we'll check it: the log tests, plus a browser walk of the Log screen
- Notes: the date is the student's local date, not the server's
```

What a good exit test looks like

Each clause names one observable thing a stranger could check: a command and whether it passed, a web address and what it shows, a number that is zero. It says nothing about how it was built. It fits on a line.

Weak	Strong

Users can log in	A new user can sign up with an email and password, sign out, sign back in, and land on the home page with their name shown
The form works	Submitting the form with an empty title shows "Title is required" and saves nothing
It's fast	The list page loads in under 2 seconds with 500 rows on the preview address
Secure	A request for the books list without being signed in is refused and returns no rows

S0 is always the skeleton

Before any feature: an empty app with its name, at a preview address, with one passing test, and the code on GitHub. S0 proves the whole path from code to a URL. Every build that skips this discovers the path is broken on the day it matters.

Modes

Mode	Means	Used when
SOLO	Build, check, prove in one session.	Most segments.
PANEL	Several expert lenses review the design before building.	A decision is contested or expensive to reverse.
REVIEW	After building, a cold reader who has seen only the card and the vision tries to find the gap between what was asked and what was delivered.	Other people's data, logins, or money appear.
CREW	The full build crew (full tier).	Logins, other people's data, payments, a rule that must never fail.

The two finish lines and the parking lot

The MVP line sits after the last card a real person needs. The Finished line sits after the last card in the vision. Every good idea that is not on the map goes in the parking lot at the bottom, so it is kept and not built yet.

The review of the map

Before any code, the map is read by lenses you do not have. On lite, one map critic wears three hats: the real user on a bad day, the engineer who has been paged at 3 a.m., and the contrarian who asks what should not be built at all. On standard and full, a panel of three to seven seats does this (see Panels). You get ranked findings with the exact edit for each, and the decisions only you can make, one at a time with a recommendation. Rulings go into `DECIDED.md` so they are never re-argued.

A worked example

Reading Log, for a fourth-grade class

Promise. Students log the books they read on the classroom iPad in under a minute. The teacher sees every student's week at a glance and can print a monthly summary for families. Nothing a student enters is visible to another student.

MVP. Students can sign in, log a book, and see their own list. The teacher can see the class for the current week. **Finished** adds the monthly family summary, reading goals, and a badge when a goal is met.

Must never. Show one student's log to another. Lose a log once it has been saved. Require a student to type an email.

The map (shortened). S0 skeleton is live · S1 the class roster and student sign-in by picture-and-PIN (REVIEW: it is children's data) · S2 students log a book and see their list (SOLO) · S3 the teacher's week-at-a-glance (PANEL: the layout is contested) · *MVP line* · S4 monthly family summary as a printable page · S5 reading goals · S6 badges · *Finished line*. Parking lot: a parent login, book covers from a public API.

5. Building a segment

You type `/drydock:build S3`, or just "next". Here is what happens, in the order you will see it, and what you do at each point.

What Drydock does

1. **Starts and restates.** It freezes the card's exit test (saves the exact words), marks the card "building", turns on the stop-gate, and tells you in plain words what you will be able to do when this is done and how it plans to get there. It makes a side branch for the work, so the main shelf stays untouched until the segment is proven.
2. **Scouts** (unless the change is one-sentence small or touches no existing code). A read-only look at what else touches the code about to change, so a fix cannot quietly break a neighbor.
3. **Writes the checks first.** One per clause. Runs them. They must fail, because the thing does not exist yet. If a check passes before the code is written, the check is wrong, and Drydock says so.
4. **Builds, in small steps.** After each step it runs the relevant check. When a check goes green, it saves a checkpoint (a commit). You will see short lines saying what it is doing, then the command it ran and the last lines of what the command printed. Not summaries.
5. **Runs every earlier segment's checks again.** A red one is fixed before anything else.
6. **Hands the work to a fresh reviewer** on a different model, with only the card, the change, and the test command. The reviewer runs the tests itself and reports gaps that affect correctness. Drydock fixes them.
7. **Proves it.** The red team attacks (see Check and prove). The running app is walked, by a robot browser on standard and full, by you on lite.
8. **Grades the frozen test by echo.** Every clause is repeated word for word with met or not and the evidence. A softened clause fails the grade automatically.
9. **Files the proof sheet, merges to the main shelf, marks the card green, updates STATE.md.**
10. **Stops and shows you.**

The stop message, and how to read it

What you can now do. In your words, with the address or the command to try it.

The exit test, clause by clause. Met or not, each with the evidence: the command and its last lines, or a screenshot path and what it shows.

What the separate check and the red team found, and what was fixed.

What is not proven, if anything. This line matters. "Not proven" is honest; "done with caveats" is not a phrase Drydock is allowed to use.

Next. "Try it. If it does what you expected, say next. If not, tell me what's off."

What you do: try it, for real, the way the person it is for would. Then one of three things:

- **"next"**: start the next segment, in a fresh session.
- **"what's off"**, in plain words: "the date is wrong when I log at 11 p.m." Drydock treats this as a new clause or a bug, fixes it, and re-proves.
- **"show me the evidence"** for any clause that is just asserted.

The stuck rule

Three honest attempts at the same failure, or a change that would widen the card, or anything that needs your data, your money, or an account it does not have: Drydock stops, says what is blocking, gives options with a recommendation, and waits. It never edits a check to make it pass. If it stops mid-segment, it says "S3 is not met" first, then what is missing.

Checkpoints and the shelf, in plain words

Git saves a checkpoint of every file. Drydock saves one every time a check goes green, on a side branch for the segment. When the segment is proven, the branch is folded into *main*, the shelf of things that work. If a segment goes badly, the shelf is untouched and the branch can be thrown away. Claude Code also keeps its own rewind points inside a session (`/rewind`) for smaller steps back.

One segment per session

Conversations get worse as they get long. After a segment is shown, hand off and start fresh. Drydock will remind you. "Keep going" always wins if you say it, but you will get better results from the discipline.

Unattended runs

Claude Code has a built-in command, `/goal`, that keeps a session working until a condition holds, judged by a separate evaluator model after every turn. For a long segment you can type the card's exit test as the goal and walk away. The stop-gate still holds a false "done" while the test command is red. Use this once you trust the flow, not on your first week.

6. Check and prove

Four separate roles look at a segment before you do. None of them is the builder. Each exists because of a specific way "done" has lied before.

Role	What it sees	What it does	What it catches
The checker	The card, the change, the test command. Not the builder's reasoning.	Runs the tests itself. Grades every clause by echo. Reads the change for bugs and for anything outside the card's scope. Checks what else the change touches.	The builder's false "done". Gaps against the card. A neighbor that broke.
The red team	The card, the list of changed files, the address to attack. Not the builder's description.	Tries empty, huge, strange, and hostile inputs. Calls the app's addresses signed out, as another user, twice fast. Attacks every rule in both directions: the bad case is refused and the good case next to it still works.	Inputs nobody tried. A rule that over-refuses the good case. Leaks.
The walker	The card's "what you can do", the address.	Uses the running app like the person it is for, with a robot browser. Screenshot or exact text as proof per task. Judges "would the person get their task done", not "did it load".	Screens that render perfectly and do the wrong thing.
The honesty audit	The draft stop message and the evidence files. Not the builder's narrative.	Traces every claim to a receipt at matching strength. Rewrites over-claims.	A summary that claims more than the evidence. "Done with caveats."

Why a different model

Same context, same blind spots. Same model, similar blind spots. Drydock runs the checker and the red team on a different model family than the builder (if the builder is Opus, the checker runs on Sonnet and the red team on Fable or Opus at high effort, and so on). This is one of the three things Drydock adds to the public methods.

What evidence looks like

Acceptable: a command in a box and the last lines of what it printed, including the exit code. A screenshot path and what the screenshot shows. The exact text read from a screen. The red team's attack bodies and what happened to each.

Not acceptable: "tests pass", "it works", "verified", "should be fine". If you see these without a receipt, say "show me the evidence".

The proof sheet

Every segment leaves `docs/reviews/<date>-S3-proof-sheet.md`: the clause table with evidence, what the checker found, what the red team tried and what held, the walk, the earlier segments' re-run, the checkpoint id, and what is not proven. A check that is not filed did not happen.

7. Panels

A panel is a set of lenses on a decision. Each seat reads the same material and hunts for what its lens sees. It is for decisions that are contested or expensive to reverse: the layout of the main screen, how sign-in works for children, whether to build a feature at all, the whole map before building starts. It is not for small changes.

The roster

The first time you convene a panel, Drydock builds a roster for your build in `docs/PANEL.md` and shows it to you. Five standing lenses: the real user on a bad day; data and privacy (whose data, who sees what, what must never leak); what breaks (the engineer at 3 a.m.); delivery cadence (is the order and size honest for your hours); the contrarian (the three ways this build dies; what should not be built). Plus domain seats your vision touches: children and consent, duty of care for health or money, offline and thumbs for phones in the field, the administrator who has to approve and support it, accessibility.

How it runs

Each seat gets one briefing file and reads only that. Seats first write their findings alone, so nobody anchors on anybody. A facilitator clusters and dedupes. On standard and full, the seats then argue across the table for a round or two, and a closing synthesis ranks everything by severity and agreement, lists the exact edits, and isolates the decisions only you can make.

Research happens before the run, in the briefing, never inside it. Seats cannot browse the web or run anything; they read.

Reading a verdict

Two files are filed: the verdict (ranked findings, exact edits, owner decisions, direction) and a plain-English companion: what the room agreed on, what it argued about and how it landed, what only you can decide. Drydock applies only the changes you accept and writes your rulings into `DECIDED.md`.

> **The panel advises. You decide.**
>
> Seats are told that candor is safe because they have no authority. Expect sharp findings. Expect disagreement. A finding is a proposal with evidence, not an order.

Cost

Lite: three seats, no deliberation, a few minutes. Standard: three seats and one round, about five agent runs. Full: five to seven seats and up to two rounds, up to twenty-five agent runs and the better part of an hour. On Pro, dynamic workflows must be switched on once in `/config` for panels to run.

8. The crew

The crew is the build loop with every check done by an independent seat, and pass or fail decided by rules a model cannot talk its way around. It is for the heaviest segments: logins, other people's data, payments, a rule that must never fail. It runs on the full tier, on cards marked CREW. It costs fifteen to twenty-five agent runs and one to three hours. It is not worth it for a screen.

What happens

1. **Pre-flight.** The exit test is frozen. If it cannot be frozen, nothing runs.
2. **Recon.** Four scouts in parallel, each blind to the others: what to fix, how any rule should be enforced, what correct behavior means in your world, what else the change touches.
3. **Build and prove, in bounded rounds.** A builder builds. In parallel, an independent scorer re-derives every claim and runs the tests itself, and the red team attacks. The scorer's last act each round is the echo-grade of the frozen test. The round passes only if the grade is clean and the red team found nothing.
4. **Verify.** Four adjudicators in parallel: the owner's lens (does this honor the vision), the checker (walks every affected screen, re-runs earlier segments), correctness, and an honesty seat whose only job is asking whether the evidence is real.
5. **Report.** A synthesizer writes a plain-English owner read and a technical receipt.
6. **Acceptance.** A relay auditor traces every claim in the owner read to a receipt and licenses exactly what may be told to you. A cold reader, shown only the card, the vision, and the change, judges whether what was delivered is what was asked. A live walker drives the deployed app.

What HALT means

If the exit test was not met, or any required seat returned nothing, the honest outcome is HALT, and that is the first word you will read. Not "done with caveats". A HALT is the crew working, not failing: it means something that would have reached you as "done" did not. You then choose: another round, a smaller card, or the safe fallback (ship with the risky part switched off).

Model lanes

The builder runs on one model family; the seats that check it run on another; a third family gives a deliberately different perspective. The crew records which models it asked for. If your plan lacks one, Claude Code substitutes, and the owner read says so, because then the model-mix defense did not hold for that run and the deterministic gates are what you are relying on.

9. Remembering

The conversation is not the memory. Files are.

File	Holds	Who reads it
`VISION.md`	What done means, in your words.	Everything that plans or judges.
`BUILD-MAP.md`	The segments, their status, their exit tests.	Build, status, wake.
`STATE.md`	Where we are: last green, in flight, next, open threads, last shipped.	Wake reads it first.
`DECIDED.md`	Settled decisions with the date and the why. Never re-argued without new evidence.	Everything, before proposing anything.
`docs/reviews/`	Proof sheets and panel verdicts.	Wake reads the newest.
`docs/handoffs/`	One note per session: what happened, decisions, state, open threads, learnings.	Wake reads the newest.

Handoff and wake

`/drydock:handoff` writes the session note, saves a snapshot, updates STATE.md, and tells you the one next move. `/drydock:wake` reads STATE.md, the map, the newest handoff and verdict, checks git, and gives you a read-back in under fifteen lines with one recommended next move. Then it waits.

When the conversation gets compressed

Claude Code compresses long conversations automatically. Right before it does, Drydock saves a snapshot (active segment, last test, recent checkpoints, the top of STATE.md). When the compressed session continues, Drydock is told to re-read STATE.md and the snapshot before doing anything. If a session feels confused after a compression, say "handoff" and start fresh.

10. Shipping

`/drydock:ship` moves what is green out into the world, in two steps, each verified by actually loading it.

1. **Preflight.** The shelf is clean, every segment's checks are green. The first time, the GitHub repository and the Netlify site are created, each with your go.
2. **Preview.** The app is built once and deployed to a private preview address. Drydock loads it and shows you the proof (the response code and a marker of the change, or a screenshot). You can send the preview link to someone to try.
3. **Live, on your yes.** Drydock asks "Preview verified at … Go live?" Only on a clear yes does it deploy to the live address, then loads that and shows the proof.
4. **Push, on your yes.** "Push the code to GitHub?" Then it records everything in STATE.md and writes a handoff.

The gate is real: a built-in check watches for publishing commands and asks you in the terminal if Drydock has not already recorded your go for that exact step. If anything fails verification, the ship stops there and shows you the failure.

Rollback: Netlify keeps every deploy; the previous one can be made live again from its dashboard in a click. The code shelf keeps every checkpoint.

11. When things go wrong

You see	What is happening	What to say or type
"Done" but it doesn't work when you try it.	A claim outran its evidence.	"Show me the evidence for each clause." Then "what's off": describe what you saw. Drydock adds a clause or a test and re-proves.
It keeps trying the same fix.	Stuck loop.	"Stop and tell me what's blocking." If it continues, "handoff", then a fresh session and `/drydock:wake`.
A test fails and it wants to change the test.	Wrong instinct. Tests are the contract.	"Fix the code, not the test." If the clause itself is wrong, say so; the card gets changed, with you, and re-frozen.
It asks you a technical question you can't answer.	It should have proposed a default.	"Recommend one and tell me what it costs if you're wrong." Then decide.
The session feels confused or repeats itself.	Context has filled or been compressed.	"Handoff." Fresh session. `/drydock:wake`.
"Usage limit reached."	Your plan's allowance for the period is spent.	Wait for the reset. Consider lite tier if this happens often: "switch Drydock to lite."
It wants to push, deploy, or publish and you didn't ask.	The owner gate caught it.	Deny in the prompt. Nothing happens. Ask what it was trying to do.
A panel verdict you disagree with.	The panel advises.	Say which findings you reject and why. They go into DECIDED.md as your ruling and are not re-argued.
Nothing seems to be happening for a long time.	A long run (panel, crew, big test) in the background.	`/workflows` shows running workflows. Or ask "what are you waiting on?"
A command "not found" (node, git, gh).	A tool is missing or not on the path.	`/drydock:setup` again; it gives the install link.
The map file looks mangled.	The card format was broken by an edit.	"Check the map format." Drydock runs the parser and fixes the shape. Section 4 shows the shape.

12. Glossary

Agent, subagent	A separate Claude with its own fresh conversation, given one job. Drydock's checker, red team, walker, and panel seats are subagents.
Branch	A side copy of the code where work happens without touching the main shelf.
Card	One segment's entry in the map: what you can do when it's done, its exit test, its mode and size.
Checkpoint, commit	A saved snapshot of every file at a moment, with a note. Drydock saves one every time a test goes green.
Compaction	Claude Code compressing a long conversation to make room. Drydock snapshots first.
Context	Everything in the current conversation. It fills up, and quality drops as it fills.
Deploy	Putting a built app at a web address. Preview addresses are private; the live address is public.
Exit test	The binary definition of done for a segment: clauses that are each met or not, with evidence.
Freeze, echo-grade	Saving the exit test's exact words at the start, and requiring the grader to repeat them exactly at the end, so a softened test is caught as a text difference.
Lane	Which model family a role runs on. The checker never shares the builder's lane.
MCP	The standard way a tool (Netlify, Supabase, a browser) is plugged into Claude Code.
Migration	A file that changes the database's shape. Applied to a test database first.
MVP	The smallest version a real person can use for the real purpose.
Plan mode	A Claude Code mode where it reads and plans but changes nothing.
Preview	A private web address for a specific version, for trying before going live.
Regression	Something that used to work and stopped. Drydock re-runs every earlier segment's checks to catch these.
Repository, repo	The project's folder with its whole checkpoint history, on your computer and on GitHub.
Stack	The tools the app is made from.
Stop-gate	Drydock's built-in check that holds a "done" message while the test command is red. Lets go after three tries, loudly.

Tier	How much machinery runs: lite, standard, full.
Workflow	A script that runs many subagents in phases. Panels and the crew are workflows.

Appendix A. Command reference

Command	Does
`/drydock:help`	Explains Drydock and the one thing to do next.
`/drydock:setup`	First-time setup: checks, four questions, memory files, companions.
`/drydock:plan`	Interview, vision, stack, map, map review.
`/drydock:build S3`	Builds one segment end to end and stops at its exit test.
`/drydock:check [S3]`	Earlier checks re-run plus a fresh reviewer, on demand.
`/drydock:prove [S3]`	Red team plus walk plus proof sheet, on demand.
`/drydock:panel [topic]`	Convenes the panel. Builds the roster the first time.
`/drydock:crew S7`	Runs the full crew on a CREW card (full tier).
`/drydock:ship`	Preview, verified; live on your yes; push on your yes.
`/drydock:wake`	Start of session: read-back and one next move.
`/drydock:handoff`	End of session: the note, a snapshot, STATE.md.
`/drydock:status`	Where the build stands.

Things you can say instead of typing commands: "build S3", "next", "where are we", "handoff", "ship it", "convene the panel on the dashboard", "show me the evidence", "stop and tell me what's blocking", "switch Drydock to standard".

Built into Claude Code and used by Drydock: `/code-review`, `/verify`, `/goal`, `/rewind`, `/workflows`, `/config`, `/plugin`.

Appendix B. The files Drydock creates

```
your-project/
├── CLAUDE.md          the project's rules; Claude reads it every session
├── VISION.md          what done means, in your words
├── BUILD-MAP.md       the segments, their status, their exit tests
├── STATE.md           where we are
├── DECIDED.md         settled decisions with the why
├── docs/
│   ├── reviews/       proof sheets, panel verdicts, crew owner-reads
```

```
│   ├── handoffs/      one note per session
│   ├── proofs/        screenshots and attack cases, by segment
│   ├── PANEL.md       the roster (once a panel has been convened)
│   └── SECRETS.md     where each key lives, never what it is
└── .drydock/
    ├── config.json    tier, your level, the stack, the commands
    ├── state.json     the active segment and the gate
    ├── regression.json  every earlier segment's checks
    ├── runs/S3/       the frozen exit test and the graded results
    └── snapshots/     saved before compaction
```

Appendix C. Where the method comes from

Read in full on October 11, 2026. Anthropic: Claude Code best practices (code.claude.com/docs/en/best-practices); the /goal and dynamic-workflows documentation; "Effective harnesses for long-running agents" (November 2025); "Harness design for long-running application development" (March 2026); "Building effective agents"; "How we built our multi-agent research system". Public methods: Superpowers (github.com/obra/superpowers); GitHub Spec Kit; BMAD-METHOD; Anthropic's feature-dev plugin; HumanLayer's Advanced Context Engineering for Coding Agents; Addy Osmani's "My LLM coding workflow going into 2026" and "Agent harness engineering"; the Ralph-loop case study at orbi.build; Thoughtworks Technology Radar vol. 34 (April 2026).

Every serious method lands on the same five moves: write the definition of done before coding, keep memory in files, have a different agent verify the builder, accept no "done" without a runnable check, and gate anything that leaves the machine. Drydock packages a protocol developed on several real products and adds three things the public methods lack: a different model family checks the builder, the summary you read is audited against receipts first, and every control traces to a specific failure with a rule for retiring it.

Drydock is MIT licensed. Source and updates: github.com/wjp3/drydock.